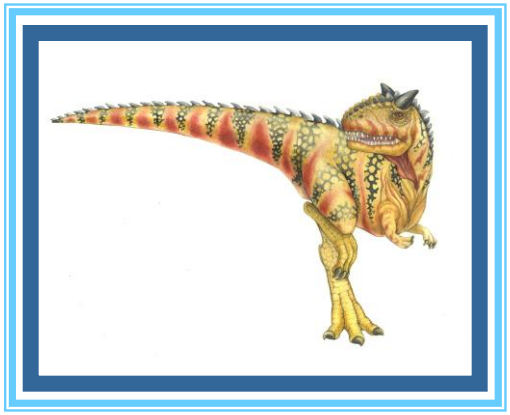
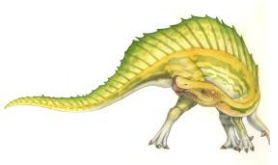
Chapter 4: Threads



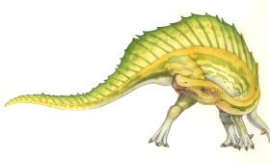
Chapter 4: Threads

- Overview
- Multithreading Models
- Thread Libraries

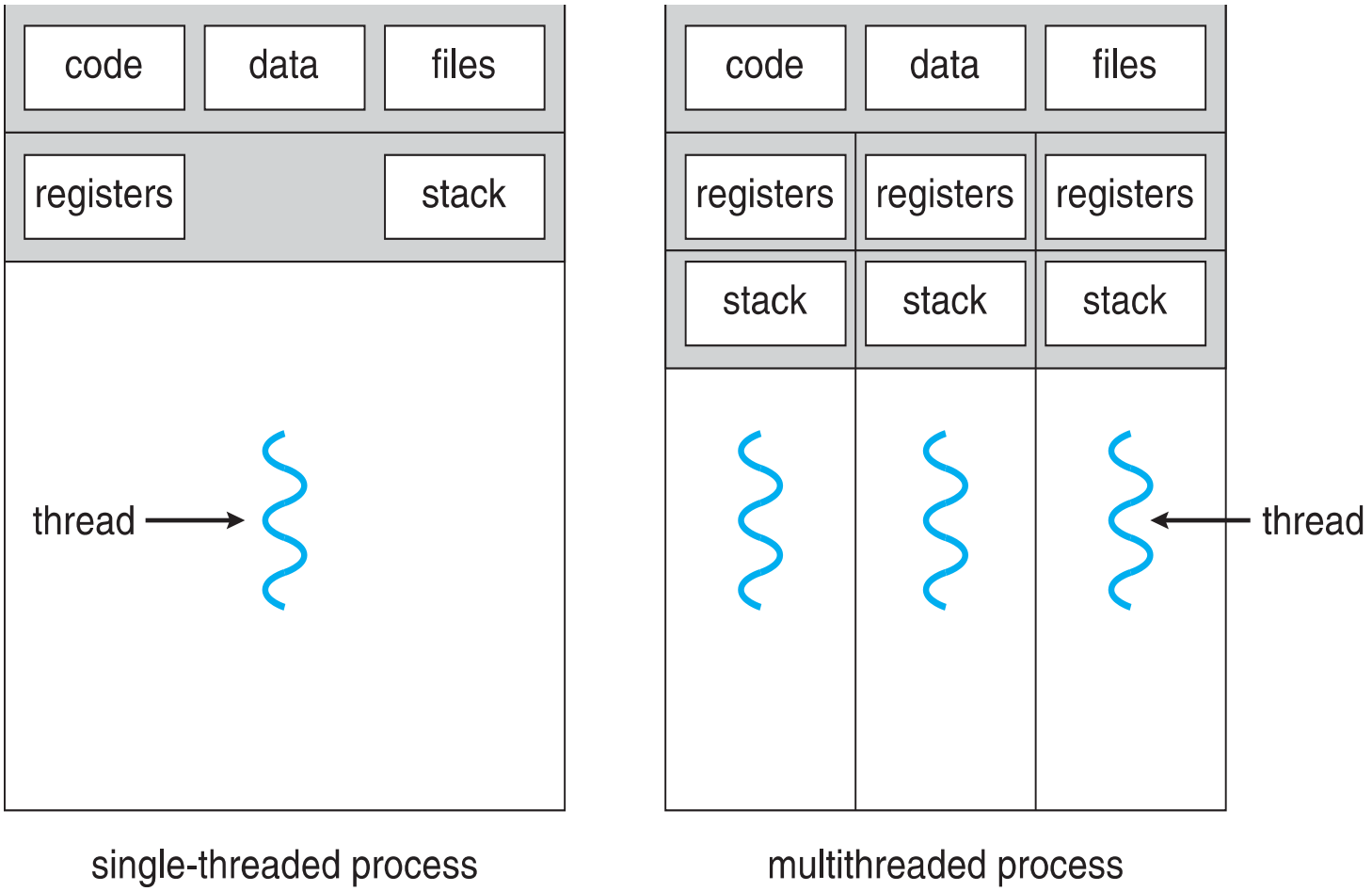


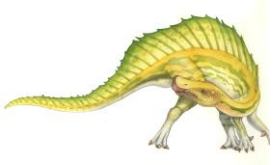
Threads - Overview

- basic unit of CPU utilization;
- comprises a thread ID, a program counter, a register set, and a stack.
- shares with other threads belonging to the same process its code section, data section, and other operating-system resources, such as open files and signals.
- A traditional (or heavyweight) process has a single thread of control.
- If a process has multiple threads of control, it can perform more than one task at a time.



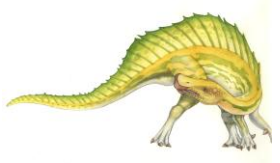
Single and Multithreaded Processes





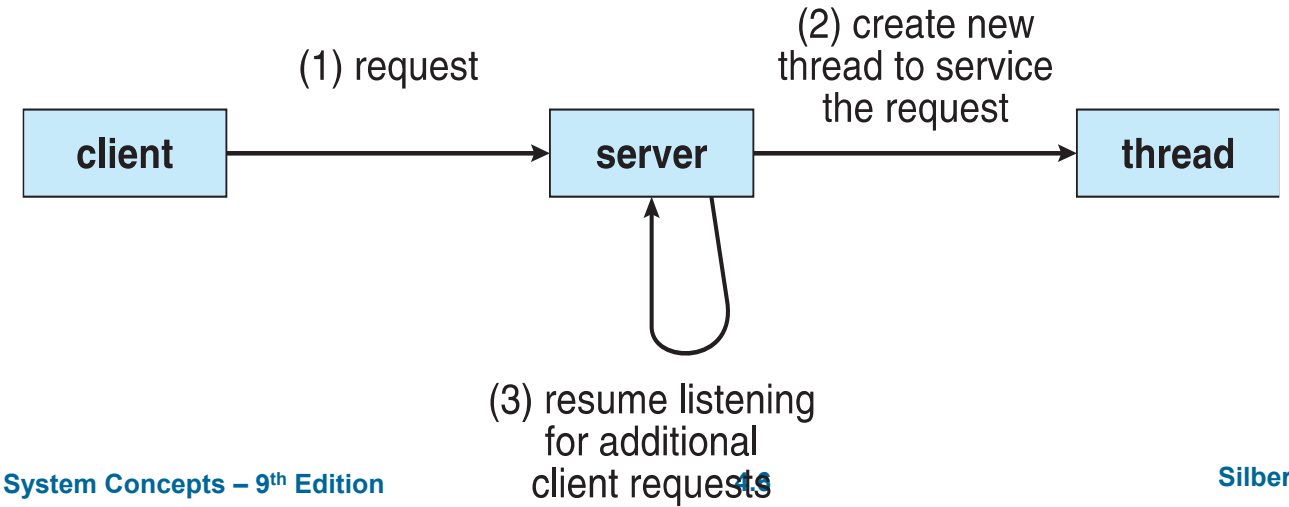
Motivation

- Most modern applications are multithreaded
- Threads run within application
- Multiple tasks with the application can be implemented by separate threads
- word processor may have a thread for
  - displaying graphics,
  - responding to keystrokes from the user
  - thread for performing spelling and grammar checking in the background
- a single application may be required to perform several similar tasks.
- Web server accepts client requests for Web pages, images, sound, and so forth.
- busy Web server may have several (perhaps thousands of) clients concurrently accessing it.
- If the Web server ran as a traditional single-threaded process, it would be able to service only one client at a time, and a client might have to wait a very long time for its request to be serviced.



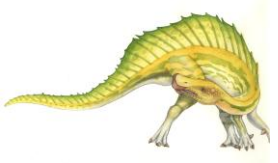
Multithreaded Server Architecture

- Instead, have the server run as a single process that accepts requests.
- When the server receives a request, it creates a separate process to service that request.
- Process creation is time consuming and resource intensive
- more efficient to use one process that contains multiple threads.
- If the Web-server process is multithreaded, the server will create a separate thread that listens for client requests.
- When a request is made, rather than creating another process, the server will create a new thread to service the request and resume listening for additional requests.



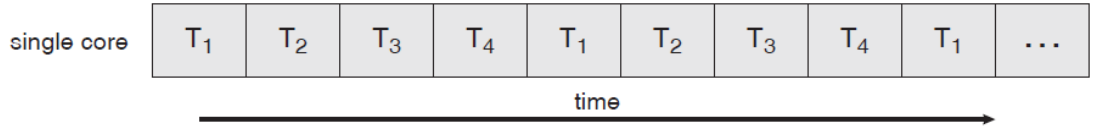
Benefits

- **Responsiveness –** may allow continued execution if part of process is blocked, especially important for user interfaces
- **Resource Sharing –** threads share resources of process by default , easier than shared memory or message passing-explicitly arranged by the programmer
- **Economy –** cheaper than process creation, thread switching-lower overhead than context switching of processes
- **Scalability –** process can take advantage of multiprocessor architectures
  - where threads may be running in parallel on different processors.
  - A single-threaded process can only run on one processor, regardless how many are available

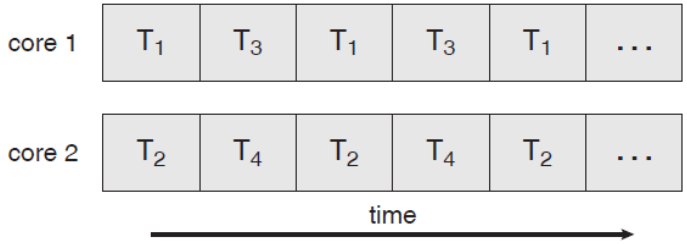


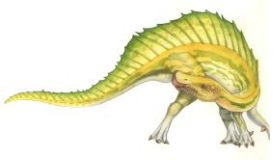
Multicore Programming

- recent trend in system design has been to place multiple computing cores on a single chip, where each core appears as a separate processor to the operating system
- Multithreaded programming provides a mechanism for more efficient use of multiple cores and improved concurrency.
- Consider an application with four threads. On a system with a single computing core, concurrency merely means that the execution of the threads will be interleaved over time ,as the processing core is capable of executing only one thread at a time.



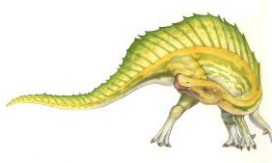
- On a system with multiple cores, however, concurrency means that the threads can run in parallel, as the system can assign a separate thread to each core
- .





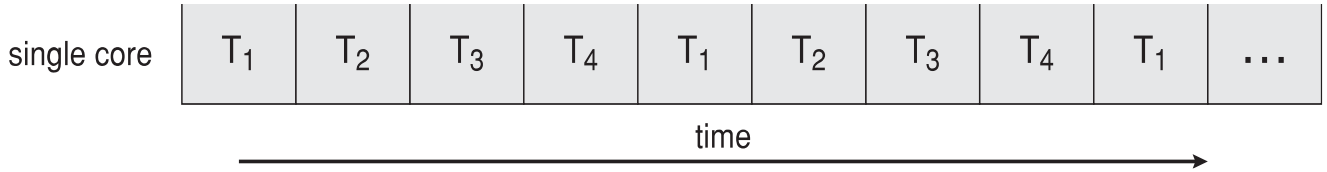
Multicore Programming

- **Multicore** or **multiprocessor** systems putting pressure on programmers, challenges include:
  - **Dividing activities** -involves examining applications to find areas that can be divided into separate, concurrent tasks and thus can run in parallel on individual cores.
  - **Balance-** ensure that the tasks perform equal work of equal value
  - **Data splitting –**the data accessed and manipulated by the tasks must be divided to run on separate cores.
  - **Data dependency -** The data accessed by the tasks must be examined for dependencies between two or more tasks
  - **Testing and debugging -**Testing and debugging concurrent programs is inherently more difficult than testing and debugging single-threaded applications - there are many different execution paths
- ***Parallelism*** implies a system can perform more than one task simultaneously
- ***Concurrency*** supports more than one task making progress
  - Single processor / core, scheduler providing concurrency

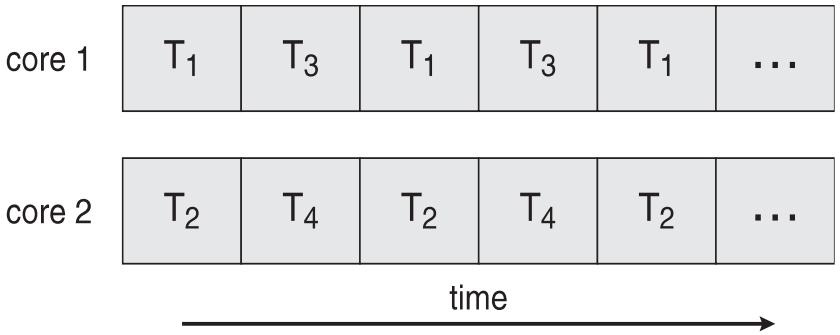


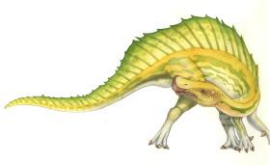
Concurrency vs. Parallelism

- **Concurrent execution on single-core system:**



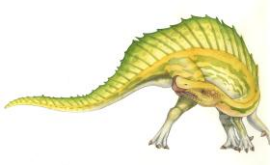
- **Parallelism on a multi-core system:**





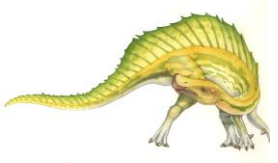
User Threads and Kernel Threads

- Support for threads -provided either at the user level, for **user threads**, or by the kernel, for **kernel threads**.
- User threads are supported above the kernel and are managed without kernel support
- Kernel threads are supported and managed directly by the operating system
- **User threads** - management done by user-level threads library
- Three primary thread libraries:
  - POSIX **Pthreads**
  - Windows threads
  - Java threads
- **Kernel threads** - Supported by the Kernel
- Examples – virtually all general purpose operating systems, including support kernel threads



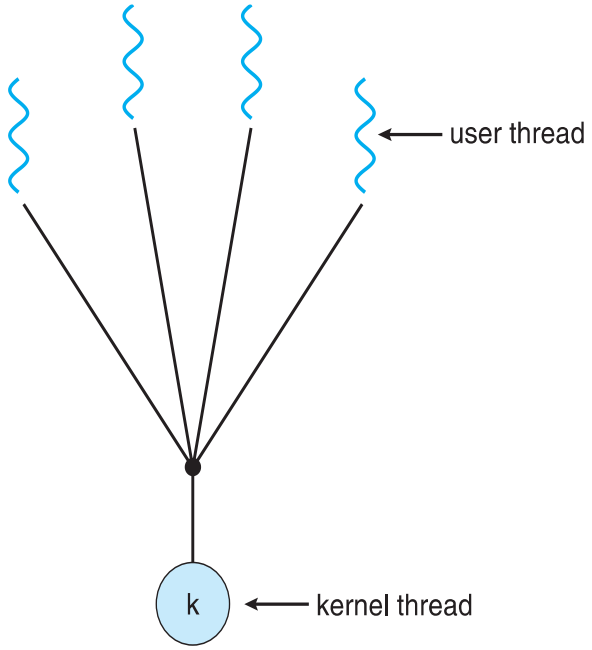
Multithreading Models

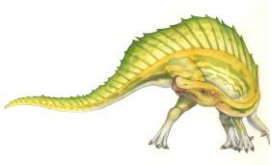
- relationship must exist between user threads and kernel threads.
  - Many-to-One
  - One-to-One
  - Many-to-Many



Many-to-One

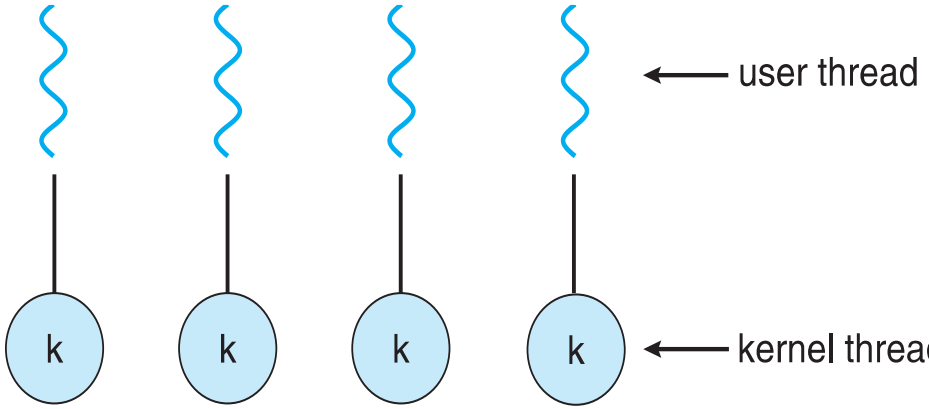
- Many user-level threads mapped to single kernel thread
- One thread blocking causes all to block
- Multiple threads may not run in parallel on multicore system because only one may be in kernel at a time
- Few systems currently use this model
- Examples:
  - **Green Threads in early Java implementations**

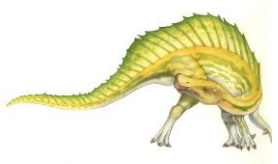




One-to-One

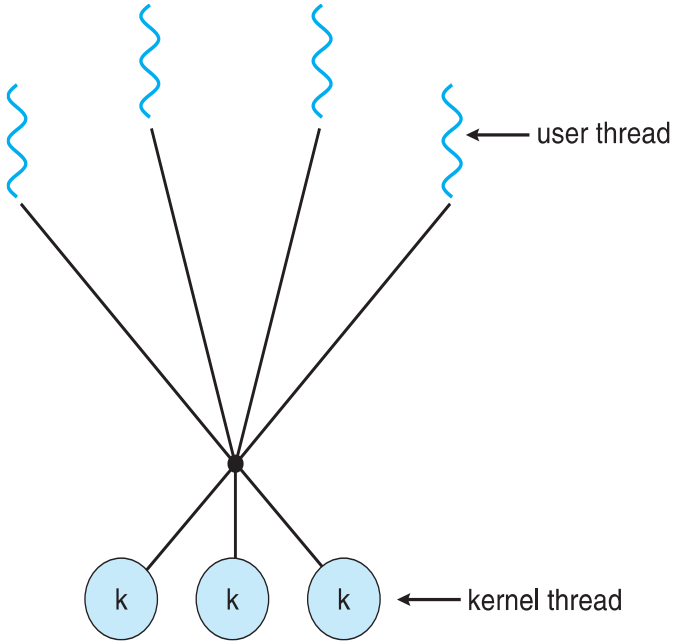
- Each user-level thread maps to kernel thread
- More concurrency than many-to-one by allowing another thread to run when a thread makes a blocking system call
- Creating a user-level thread creates a kernel thread
- Because the overhead of creating kernel threads can burden the performance of an application, most implementations of this model restrict the number of threads per process supported by the system
- Examples
  - POSIX(**The Portable Operating System Interface** is a family of standards specified by the IEEE Computer Society for maintaining compatibility between operating systems) threads in Linux

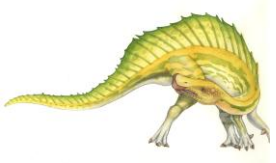




Many-to-Many Model

- multiplexes many user-level threads to a smaller or equal number of kernel threads
- Allows the operating system to create a sufficient number of kernel threads
- The number of kernel threads may be specific to either a particular application or a particular machine
- Corresponding kernel threads can run in parallel on a multiprocessor.
- when a thread performs a blocking system call, the kernel can schedule another thread for execution
- Solaris prior to version 9
- Windows NT

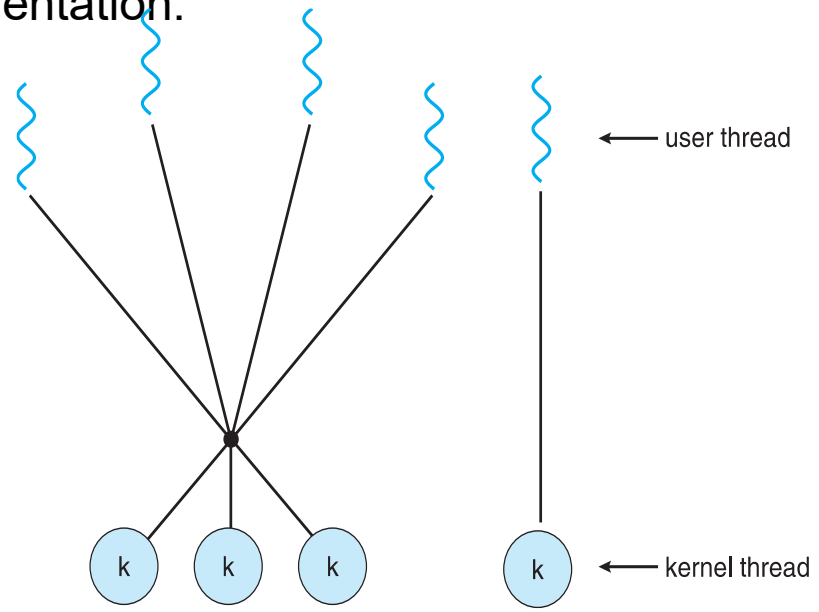


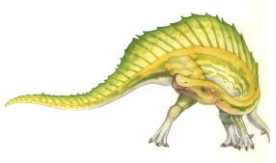


Two-level Model

- Similar to M:M, where some user threads are bound to kernel threads, while others are multiplexed
- Offers flexibility and efficiency.
- Allows both bound and unbound threads.
- Increased complexity in implementation.

- Examples
  - IRIX
  - HP-UX
  - Tru64 UNIX
  - Solaris 8 and earlier



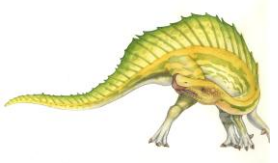


Thread Libraries

- **Thread library** provides programmer with API for creating and managing threads
- Two primary ways of implementing
  - Library entirely in **user space**
    - All code and data structures for the library exist in user space.
    - invoking a function in the library results in a local function call in user space and not a system call
  - **Kernel-level l**ibrary supported by the OS
    - code and data structures for the library exist in kernel space.
    - Invoking a function in the API for the library typically results in a system call to the kernel.
- Three main thread libraries
  - POSIX Pthreads -provided as either a user- or kernel-level library.
  - Win32 -kernel-level library available on Windows systems.
  - Java.- Java thread API allows threads to be created and managed directly in Java programs

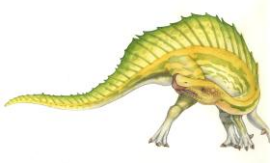
Pthreads

- May be provided either as user-level or kernel-level
- A POSIX standard (IEEE 1003.1c) API for thread creation and synchronization
- ***Specification***, not ***implementation***
- API specifies behavior of the thread library, implementation is up to development of the library
- Common in UNIX operating systems (Solaris, Linux, Mac OS X)



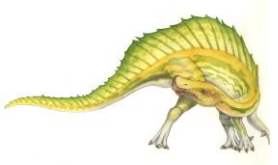
Java Threads

- Java threads are managed by the JVM
- Typically implemented using the threads model provided by underlying OS



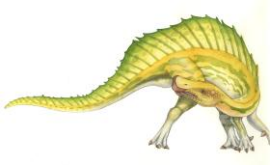
Acknowledgement

- Thie slides are borrowed from the ppt of Operating Systems Concepts 9Th Edition By Silberschatz, Galvin and Gagne with some modifications done



Reference

- A. Silberschatz, P. B. Galvin and G. Gagne, *Operating System Concepts*, (9e), Wiley and Sons (Asia) Pvt. Ltd, 2016.



End of Chapter 4